Step 1 – Create Database

Explanation

The CREATE DATABASE command creates a new database named **bookflow_dbms**. The USE command selects this database so that all subsequent SQL commands are executed within it. A successful execution confirms that the database has been created and activated.

```
mysql> CREATE DATABASE bookflow_dbms;
Query OK, 1 row affected (0.01 sec)
```

Step 2 – Create Books Table

Explanation

The **Books** table is created to store book information. It includes a **PRIMARY KEY** (book_id) to uniquely identify each book, a **NOT NULL** constraint on the title, a **UNIQUE** constraint on ISBN to prevent duplicate entries, and a **CHECK** constraint to ensure valid publication years.

```
mysql> USE bookflow_dbms;
Database changed
```

Step 3 – Insert Book Records

Explanation

Three sample book records are inserted into the Books table using a single INSERT INTO statement. The successful execution confirms that all records satisfy the defined constraints.

```
mysql> CREATE TABLE Books (
    -> book_id INT PRIMARY KEY,
    -> title VARCHAR(100) NOT NULL,
    -> isbn VARCHAR(20) UNIQUE,
    -> published_year INT CHECK (published_year < 2027)
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Step 4 – Display Books Table

Explanation

The SELECT * FROM Books; query displays all the records stored in the Books table. This verifies that the data has been inserted successfully.

```
mysql> INSERT INTO Books (book_id, title, isbn, published_year) VALUES
    -> (1, 'The Great Gatsby', '9780743273565', 1925),
    -> (2, 'To Kill a Mockingbird', '9780061120084', 1960),
    -> (3, '1984', '9780451524935', 1949);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Step 5 – Create Members Table

Explanation

The **Members** table is created to maintain library member information. Each member has a unique member_id, while the email column is marked as **UNIQUE** to avoid duplicate registrations.

```
mysql> SELECT * FROM Books;
+---------+----------------------+---------------+----------------+
| book_id | title                | isbn          | published_year |
+---------+----------------------+---------------+----------------+
|       1 | The Great Gatsby     | 9780743273565 |           1925 |
|       2 | To Kill a Mockingbird | 9780061120084 |           1960 |
|       3 | 1984                 | 9780451524935 |           1949 |
+---------+----------------------+---------------+----------------+
3 rows in set (0.00 sec)
```

Step 6 – Display Members Table

Explanation

The SELECT * FROM Members; statement retrieves all member records. The output confirms that the member details have been stored correctly.

```
mysql> CREATE TABLE Members (
    -> member_id INT PRIMARY KEY,
    -> full_name VARCHAR(100),
    -> email VARCHAR(100) UNIQUE
    -> );
Query OK, 0 rows affected (0.05 sec)
```

Step 7 – Create Loans Table

Explanation

The **Loans** table connects Members and Books using **FOREIGN KEY** constraints. It stores borrowing transactions while maintaining referential integrity between the related tables.

```
mysql> SELECT * FROM Members;
+-----------+---------------+-------------------------+
| member_id | full_name     | email                   |
+-----------+---------------+-------------------------+
|       101 | John Smith    | john.smith@email.com    |
|       102 | Emma Wilson   | emma.wilson@email.com   |
|       103 | Michael Brown | michael.brown@email.com |
+-----------+---------------+-------------------------+
3 rows in set (0.00 sec)
```

Step 8 – Insert Loan Records

Explanation

Ten borrowing transactions are inserted into the Loans table. Since all referenced members and books already exist, every record satisfies the foreign key constraints.

```
mysql> CREATE TABLE Loans (
    -> loan_id INT PRIMARY KEY,
    -> member_id INT,
    -> book_id INT,
    -> loan_date DATE,
    -> FOREIGN KEY (member_id) REFERENCES Members(member_id),
    -> FOREIGN KEY (book_id) REFERENCES Books(book_id)
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Step 9 – Display Loans Table

Explanation

The SELECT * FROM Loans; query displays all borrowing records, confirming that the loan information has been inserted successfully.

```
mysql> INSERT INTO Loans (loan_id, member_id, book_id, loan_date) VALUES
    -> (1, 101, 1, '2025-01-05'),  (2, 102, 2, '2025-01-08'),
    -> (3, 103, 3, '2025-01-10'),  (4, 101, 2, '2025-02-01'),
    -> (5, 102, 1, '2025-02-05'),  (6, 103, 2, '2025-02-12'),
    -> (7, 101, 3, '2025-03-01'),  (8, 102, 3, '2025-03-07'),
    -> (9, 103, 1, '2025-03-15'),  (10, 101, 1, '2025-04-01');
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

Step 10 – JOIN Query

Explanation

An **INNER JOIN** combines the Books, Members, and Loans tables to display readable information instead of numeric IDs. The output shows the member name together with the title of the borrowed book.

```
mysql> SELECT * FROM Loans;
+---------+-----------+---------+------------+
| loan_id | member_id | book_id | loan_date  |
+---------+-----------+---------+------------+
|       1 |       101 |       1 | 2025-01-05 |
|       2 |       102 |       2 | 2025-01-08 |
|       3 |       103 |       3 | 2025-01-10 |
|       4 |       101 |       2 | 2025-02-01 |
|       5 |       102 |       1 | 2025-02-05 |
|       6 |       103 |       2 | 2025-02-12 |
|       7 |       101 |       3 | 2025-03-01 |
|       8 |       102 |       3 | 2025-03-07 |
|       9 |       103 |       1 | 2025-03-15 |
|      10 |       101 |       1 | 2025-04-01 |
+---------+-----------+---------+------------+
10 rows in set (0.00 sec)
```

Step 11 – GROUP BY Query

Explanation

The GROUP BY clause groups books based on their publication year. The COUNT() function calculates how many books belong to each year, producing a summarized report.

```
mysql> SELECT m.full_name AS Member_Name, b.title AS Book_Title
    -> FROM Loans l
    -> INNER JOIN Members m ON l.member_id = m.member_id
    -> INNER JOIN Books b ON l.book_id = b.book_id;
+--------------+----------------------+
| Member_Name  | Book_Title           |
+--------------+----------------------+
| John Smith   | The Great Gatsby     |
| John Smith   | To Kill a Mockingbird |
| John Smith   | 1984                 |
| John Smith   | The Great Gatsby     |
| Emma Wilson  | To Kill a Mockingbird |
| Emma Wilson  | The Great Gatsby     |
| Emma Wilson  | 1984                 |
| Michael Brown | 1984                |
| Michael Brown | To Kill a Mockingbird |
| Michael Brown | The Great Gatsby    |
+--------------+----------------------+
10 rows in set (0.01 sec)
```

Step 12 – Create Donation History Table

Explanation

The **Donation_History** table records details of donated books. It maintains a relationship with the Books table through a foreign key.

```
mysql> SELECT published_year, COUNT(book_id) AS Total_Books
    -> FROM Books
    -> GROUP BY published_year
    -> ORDER BY published_year;
+----------------+-------------+
| published_year | Total_Books |
+----------------+-------------+
|           1925 |           1 |
|           1949 |           1 |
|           1960 |           1 |
+----------------+-------------+
3 rows in set (0.01 sec)
```

Step 13 – Start Transaction

Explanation

The START TRANSACTION statement begins an SQL transaction. Multiple database operations are treated as a single unit to maintain data consistency.

```
mysql> CREATE TABLE Donation_History (
    ->     donation_id INT PRIMARY KEY,
    ->     book_id INT,
    ->     donor_name VARCHAR(100),
    ->     donation_date DATE,
    ->     FOREIGN KEY (book_id) REFERENCES Books(book_id)
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Step 14 – Insert New Book

Explanation

A new book (**Animal Farm**) is inserted into the Books table as part of the ongoing transaction.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)
```

Step 15 – Insert Donation Record

Explanation

The donated book information is simultaneously recorded in the Donation_History table. Both operations belong to the same transaction.

```
mysql> INSERT INTO Books (book_id, title, isbn, published_year)
    -> VALUES (4, 'Animal Farm', '9780451526342', 1945);
Query OK, 1 row affected (0.00 sec)
```

Step 16 – Commit Transaction

Explanation

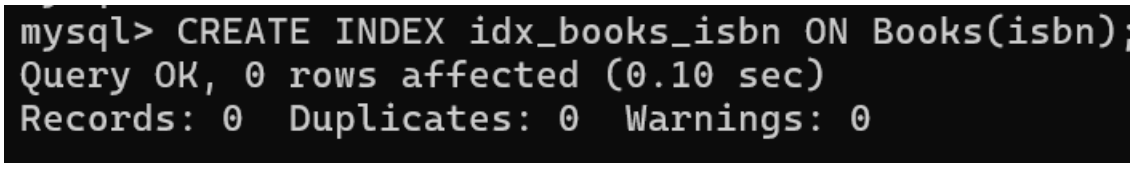
The COMMIT statement permanently saves all changes made during the transaction. If any operation had failed, a rollback could have restored the database to its previous state.

```
mysql> INSERT INTO Donation_History (donation_id, book_id, donor_name, donation_date)
    -> VALUES (1, 4, 'Raj Kumar', CURDATE());
Query OK, 1 row affected (0.00 sec)
```

Step 17 – Create Index

Explanation

An index named **idx_books_isbn** is created on the ISBN column. Indexing improves search performance by allowing MySQL to locate records more efficiently.



Step 18 – Search Using ISBN

Explanation

The SELECT query searches for a book using its ISBN value. Since an index exists on the ISBN column, the lookup is faster than performing a full table scan.

```
mysql> SELECT * FROM Books WHERE isbn = '9780451524935';
+---------+-------+---------------+----------------+
| book_id | title | isbn          | published_year |
+---------+-------+---------------+----------------+
|       3 | 1984  | 9780451524935 |           1949 |
+---------+-------+---------------+----------------+
1 row in set (0.00 sec)
```